部署編集ダイアログUI設計書

メタデータ

title: 部署編集ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-01-23
author: Development Team
tags: [system-admin, department, dialog, edit]

関連ドキュメント

- 部署管理メイン画面
- 部署追加ダイアログ
- マスタ管理API仕様
- 部署テーブルDDL
- バックエンド実装計画
- UIポリシー

---

2. 画面レイアウト

2.1. 画面イメージ


[部署編集ダイアログ画面イメージ]

---

3. 概要

3.1. 目的

システム管理者が既存の部署情報を編集するためのダイアログを定義します。部署名とアクティブ状態を変更できます。部署CDは変更不可（主キーのため）。

3.2. 利用権限

- **System Admin（システム管理者）**: 使用可能

---

2. ダイアログ構成

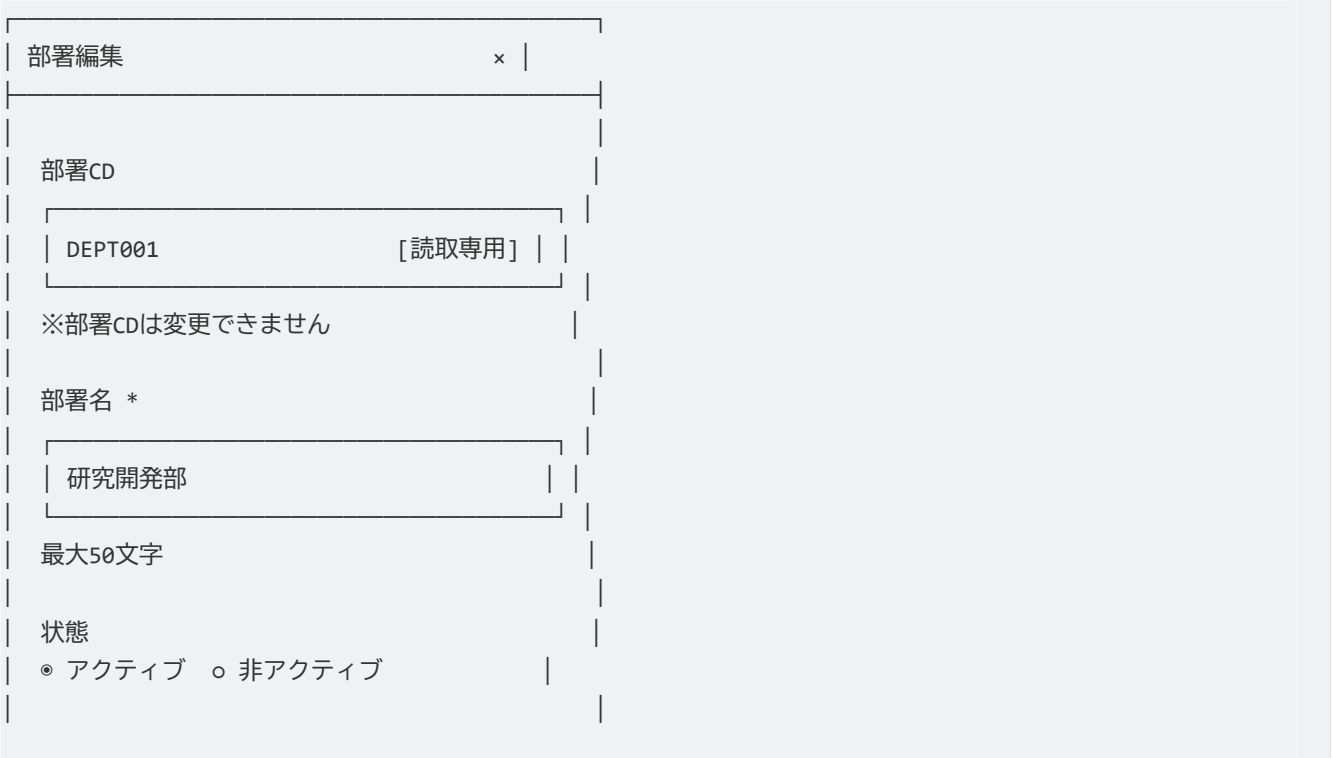
2.1. ダイアログ概要

表示トリガー: 部署管理画面の「編集」ボタンクリック

モーダルタイプ: Modal

サイズ: 中（600px）

2.2. レイアウト



```
├──────────────────────────────────────────┤
│                    [キャンセル] [保存]    │
└──────────────────────────────────────────┘
```

---

3. 入力項目

3.1. 部署CD（表示のみ）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「部署CD」
型	text
読み取り専用	○（disabled）
説明	「※部署CDは変更できません」

実装例:

```
<Form.Item
  label="部署CD"
  name="departmentCd"
  extra="※部署CDは変更できません"
>
  <Input disabled />
</Form.Item>
```

3.2. 部署名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「部署名」

プレースホルダー	「部署名を入力」
型	text
必須	○
最小文字数	1文字
最大文字数	50文字

バリデーション:

```
{
  required: true,
  message: '部署名を入力してください'
},
{
  min: 1,
  max: 50,
  message: '部署名は1〜50文字で入力してください'
}
```

3.3. 状態（必須）

コンポーネント: `<Form.Item>` + `<Radio.Group>`

仕様:

項目	値
ラベル	「状態」
選択肢	アクティブ / 非アクティブ
必須	○

実装例:

```
<Form.Item
  label="状態"
  name="isActive"
  rules={[{ required: true, message: '状態を選択してください' }]}
>
  <Radio.Group>
    <Radio value={true}>アクティブ</Radio>
    <Radio value={false}>非アクティブ</Radio>
```

```
    </Radio.Group>
  </Form.Item>
```

---

4. アクションボタン

4.1. キャンセルボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「キャンセル」
タイプ	`default`
配置	フッター左側

動作:

1. 入力内容を破棄
2. ダイアログを閉じる
3. 親画面に戻る

4.2. 保存ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「保存」
タイプ	`primary`
配置	フッター右側
ローディング	API呼び出し中は `true`
無効化	バリデーションエラー時

動作:

1. フォームバリデーション実行
2. 変更があるか確認
3. API呼び出し: `PUT /api/v1/master/department/:departmentCd`
4. 成功時:
   - 成功メッセージ表示（Toast）
   - ダイアログを閉じる
   - 親画面の一覧を再取得
5. 失敗時: エラーメッセージ表示

---

5. データの初期化

5.1. ダイアログ表示時

編集対象の部署データを取得してフォームに設定:

```
useEffect(() => {
  if (open && departmentCd) {
    fetchDepartmentData(departmentCd);
  }
}, [open, departmentCd]);

const fetchDepartmentData = async (cd: string) => {
  try {
    const response = await apiClient.get(
      `/api/v1/master/department/${cd}`,
      {
        headers: {
          'x-organization-id': organizationId
        }
      }
    );

    if (response.data.success) {
      form.setFieldsValue({
        departmentCd: response.data.data.departmentCd,
        departmentName: response.data.data.departmentName,
```

```
        isActive: response.data.data.isActive
      });
    }
  } catch (error) {
    message.error('部署情報の取得に失敗しました');
  }
};
```

---

6. APIリクエスト仕様

6.1. データ取得エンドポイント

`GET /api/v1/master/department/:departmentCd`

リクエストヘッダー:

```
Authorization: Bearer {jwt_token}
x-organization-id: {organization_id}
```

レスポンス（成功時）:

```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "departmentCd": "DEPT001",
    "departmentName": "研究開発部",
    "isActive": true,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T10:30:00.000Z"
  },
  "meta": {
    "timestamp": "2026-01-23T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```

6.2. 更新エンドポイント

`PUT /api/v1/master/department/:departmentCd`

リクエストヘッダー:

```
Content-Type: application/json
Authorization: Bearer {jwt_token}
x-organization-id: {organization_id}
```

リクエストボディ:

```
{
  "departmentName": "研究開発第一部",
  "isActive": true
}
```

レスポンス（成功時）:

```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "departmentCd": "DEPT001",
    "departmentName": "研究開発第一部",
    "isActive": true,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T10:35:00.000Z"
  },
  "meta": {
    "timestamp": "2026-01-23T10:35:00.000Z",
    "requestId": "req_abc456"
  }
}
```

レスポンス（エラー時）:

```
{
  "success": false,
```

```
  "error": {
    "code": "DEPARTMENT_NOT_FOUND",
    "message": "部署が見つかりません",
    "details": []
  },
  "meta": {
    "timestamp": "2026-01-23T10:35:00.000Z",
    "requestId": "req_abc456"
  }
}
```

---

7. エラーメッセージ

7.1. バリデーションエラー

エラー条件	メッセージ
部署名が空	「部署名を入力してください」
部署名が長すぎる	「部署名は1〜50文字で入力してください」
状態が未選択	「状態を選択してください」

7.2. APIエラー

エラーコード	メッセージ
`DEPARTMENT_NOT_FOUND`	「部署が見つかりません」
`VALIDATION_ERROR`	「入力内容に誤りがあります」
`UNAUTHORIZED`	「この操作を実行する権限がありません」
`INTERNAL_SERVER_ERROR`	「サーバーエラーが発生しました。しばらく経ってから再試行してください」

---

8. 実装例（TypeScript + Next.js + Ant Design）

```
// frontend/src/components/SystemAdmin/DepartmentEditDialog.tsx
'use client';
```

```tsx
import React, { useState, useEffect } from 'react';
import { Modal, Form, Input, Radio, Button, message } from 'antd';
import apiClient from '@/utils/apiClient';

interface DepartmentEditDialogProps {
  open: boolean;
  onClose: () => void;
  onSuccess: () => void;
  departmentCd: string | null;
  organizationId: string;
}

interface DepartmentFormValues {
  departmentCd: string;
  departmentName: string;
  isActive: boolean;
}

export default function DepartmentEditDialog({
  open,
  onClose,
  onSuccess,
  departmentCd,
  organizationId
}: DepartmentEditDialogProps) {
  const [form] = Form.useForm();
  const [isLoading, setIsLoading] = useState(false);
  const [isFetching, setIsFetching] = useState(false);

  // データ取得
  useEffect(() => {
    if (open && departmentCd) {
      fetchDepartmentData(departmentCd);
    }
  }, [open, departmentCd]);

  const fetchDepartmentData = async (cd: string) => {
    setIsFetching(true);
    try {
      const response = await apiClient.get(
        `/api/v1/master/department/${cd}`,
        {
          headers: {
            'x-organization-id': organizationId
          }
        }
      );

      if (response.data.success) {
```

```
      const data = response.data.data;
      form.setFieldsValue({
        departmentCd: data.departmentCd,
        departmentName: data.departmentName,
        isActive: data.isActive
      });
    }
  } catch (error: any) {
    const errorMessage =
      error.response?.data?.error?.message || '部署情報の取得に失敗しました';
    message.error(errorMessage);
    onClose();
  } finally {
    setIsFetching(false);
  }
};

const handleSubmit = async (values: DepartmentFormValues) => {
  if (!departmentCd) return;

  setIsLoading(true);
  try {
    const response = await apiClient.put(
      `/api/v1/master/department/${departmentCd}`,
      {
        departmentName: values.departmentName,
        isActive: values.isActive
      },
      {
        headers: {
          'x-organization-id': organizationId
        }
      }
    );

    if (response.data.success) {
      message.success('部署を更新しました');
      form.resetFields();
      onSuccess();
      onClose();
    }
  } catch (error: any) {
    const errorMessage =
      error.response?.data?.error?.message || '部署の更新に失敗しました';
    message.error(errorMessage);
  } finally {
    setIsLoading(false);
  }
};
```

```
  const handleCancel = () => {
    form.resetFields();
    onClose();
  };

  return (
    <Modal
      title="部署編集"
      open={open}
      onCancel={handleCancel}
      footer={null}
      width={600}
      destroyOnClose
    >
      <Form
        form={form}
        layout="vertical"
        onFinish={handleSubmit}
      >
        {/* 部署CD（読み取り専用） */}
        <Form.Item
          label="部署CD"
          name="departmentCd"
          extra="※部署CDは変更できません"
        >
          <Input disabled />
        </Form.Item>

        {/* 部署名 */}
        <Form.Item
          label="部署名"
          name="departmentName"
          rules={[
            { required: true, message: '部署名を入力してください' },
            { min: 1, max: 50, message: '部署名は1〜50文字で入力してください' }
          ]}
          extra="最大50文字"
        >
          <Input placeholder="部署名を入力" maxLength={50} />
        </Form.Item>

        {/* 状態 */}
        <Form.Item
          label="状態"
          name="isActive"
          rules={[{ required: true, message: '状態を選択してください' }]}
        >
          <Radio.Group>
            <Radio value={true}>アクティブ</Radio>
            <Radio value={false}>非アクティブ</Radio>
```

```
          </Radio.Group>
        </Form.Item>

        {/* フッター */}
        <Form.Item style={{ marginBottom: 0, marginTop: 24 }}>
          <div style={{ display: 'flex', justifyContent: 'flex-end', gap: 8 }}>
            <Button onClick={handleCancel}>キャンセル</Button>
            <Button
              type="primary"
              htmlType="submit"
              loading={isLoading || isFetching}
              disabled={isLoading || isFetching}
            >
              保存
            </Button>
          </div>
        </Form.Item>
      </Form>
    </Modal>
  );
}
```

---

9. 変更検知と最適化

9.1. 変更なし判定

フォームの値が初期値と同じ場合、APIを呼び出さずに確認メッセージを表示:

```
const handleSubmit = async (values: DepartmentFormValues) => {
  // 初期値と比較
  const initialValues = form.getFieldsValue();
  const hasChanged =
    values.departmentName !== initialValues.departmentName ||
    values.isActive !== initialValues.isActive;

  if (!hasChanged) {
    message.info('変更がありません');
    return;
  }
```

```
  // API呼び出し...
};
```

9.2. 確認ダイアログ

重要な変更（非アクティブ化）の場合、確認ダイアログを表示:

```
import { Modal } from 'antd';

const handleSubmit = async (values: DepartmentFormValues) => {
  // 非アクティブ化の場合は確認
  if (!values.isActive && initialValues.isActive) {
    Modal.confirm({
      title: '部署を非アクティブ化しますか？',
      content: '非アクティブにすると、この部署を使用できなくなります。',
      okText: 'はい',
      cancelText: 'いいえ',
      onOk: () => submitUpdate(values)
    });
  } else {
    submitUpdate(values);
  }
};
```

---

10. テスト要件

10.1. 単体テスト

テストケース

- データ取得機能のテスト
- バリデーション機能のテスト
- フォーム送信処理のテスト
- 変更検知機能のテスト

10.2. 統合テスト

テストシナリオ

1. 正常な部署編集
2. 部署名のみ変更
3. 状態のみ変更
4. 変更なしで保存
5. 存在しない部署の編集
6. 非アクティブ化の確認ダイアログ

---

11. アクセシビリティ

11.1. キーボード操作

- Tab: 次の入力項目へフォーカス移動
- Shift + Tab: 前の入力項目へフォーカス移動
- Enter: フォーム送信
- Escape: ダイアログを閉じる

11.2. スクリーンリーダー対応

- 読み取り専用フィールドに `aria-readonly="true"` を設定
- エラーメッセージを `role="alert"` で通知
- 必須項目を `aria-required="true"` で明示

---

12. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-01-23	初版作成	Development Team